

Top-Down Design with Functions

SDC PPWC CSE 3544

**Department of Computer Science & Engineering
ITER, Siksha 'O' Anusandhan Deemed To Be University
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030**

Book(s)

Text Book(s)



B. M. Harwani

Practical C Programming

Packt Publishing Ltd.

Reference Book(s)



Jeri R Hanly, & Elliot B. Koffman

Problem Solving and Program Design in C

Seventh Edition

Pearson Education



Kay A. Robbins, & Steve Robbins

UnixTM Systems Programming

Communications, concurrency, and Treads

Pearson Education

Talk Flow

- 1 Introduction
- 2 Library Functions
- 3 Top-Down Design and Structure Charts
- 4 User Defined Functions
- 5 Quick-check Exercises

Function

A function is a block of code which only runs when it is called and performs a task. We can categorize C functions into two categories, i.e., Library functions and User defined functions. Some examples of these two types of functions are given in the following table.

Table 1: Functions

Library functions	User defined functions
<i>printf()</i>	<i>sum()</i>
<i>scanf()</i>	<i>check_digit()</i>
<i>sqrt()</i>	<i>check_odd_even()</i>
<i>abs()</i>	<i>check_prime()</i>

Library Functions

Library Functions (predefined functions and code reuse)

Library functions are built-in functions that are grouped together and placed in a common location called library.

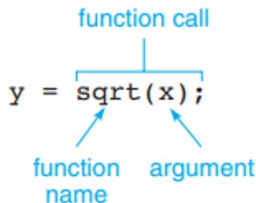


Figure 1: Example of library function

Function sqrt as a "Black Box"

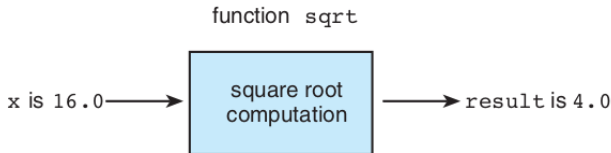


Figure 2: Function as a Black Box

Library Functions (contd.) I

Function	Include File	Function Prototype	Description
abs	stdlib.h	int abs(int n);	Calculates the absolute value of an integer argument n
sin	math.h	double sin(double x);	Calculates the sine of x.
cos	math.h	double cos(double x);	Calculates the cosine of x.
tan	math.h	double tan(double x);	Calculates the tangent of x.
ceil	math.h	double ceil(double x);	Calculates the double value representing the smallest integer that is greater than or equal to x.
floor	math.h	double floor(double x);	Calculates the floating-point value representing the largest integer less than or equal to x.
exp	math.h	double exp(double x);	Calculates the exponential function of a floating-point argument x.
pow	math.h	double pow(double x, double y);	Calculates the value x to the power y.
sqrt	math.h	double sqrt(double x);	Calculates the square root of x.
clock	time.h	clock_t clock(void);	Returns the processor time that has elapsed since the job was started.
ctime	time.h	char *ctime(const time_t *time);	Converts time to a character string.
difftime	time.h	double difftime(time_t time2, time_t time1);	Computes the difference between time2 and time1.
isalpha	ctype.h	int isalpha(int c);	Tests if c is alphabetic.
isblank	ctype.h	int isblank(int c);	Tests if c is a blank or tab character.
isdigit	ctype.h	int isdigit(int c);	Tests if c is a decimal digit.
islower	ctype.h	int islower(int c);	Tests if c is a lowercase letter.
isupper	ctype.h	int isupper(int c);	Tests if c is an uppercase letter.
printf	stdio.h	int printf(const char *format-string, arg-list);	Formats and prints characters and values to stdout.
scanf	stdio.h	int scanf(const char *format-string, arg-list);	Reads data from stdin into locations given by arg-list.

Library Functions (contd.) II

fprintf	stdio.h	int fprintf(FILE *stream, const char *format-string, arg-list);	Formats and prints characters and values to the output stream.
fscanf	stdio.h	int fscanf(FILE *stream, const char *format-string, arg-list);	Reads data from stream into locations given by arg-list.
sprintf	stdio.h	int sprintf(char *buffer, const char *format-string, arg-list);	Formats and stores characters and values in buffer.
sscanf	stdio.h	int sscanf(const char *buffer, const char *format, arg-list);	Reads data from buffer into the locations given by arg-list.
getchar	stdio.h	int getchar(void);	Reads a single character from stdin.
putchar	stdio.h	int putchar(int c);	Prints c to stdout.
getc	stdio.h	int getc(FILE *stream);	Reads a single character from the input stream.
putc	stdio.h	int putc(int c, FILE *stream);	Prints c to the output stream.
gets	stdio.h	char *gets(char *buffer);	Reads a string from stdin, and stores it in buffer.
puts	stdio.h	int puts(const char *string);	Prints a string to stdout.
fgets	stdio.h	char *fgets(char *string, int n, FILE *stream);	Reads a string from the input stream.
fputs	stdio.h	int fputs(const char *string, FILE *stream);	Copies a string to the output stream.
fopen	stdio.h	FILE *fopen(const char *filename, const char *mode);	Opens the specified file.
fread	stdio.h	size_t fread(void *buffer, size_t size, size_t count, FILE *stream);	Reads up to count items of size length from the input stream, and stores them in buffer.
fwrite	stdio.h	size_t fwrite(const void *buffer, size_t size, size_t count, FILE *stream);	Writes up to count items of size length from buffer to stream.
fseek	stdio.h	int fseek(FILE *stream, long int offset, int origin);	Changes the current file position associated with stream to a new location.
ftell	stdio.h	long int ftell(FILE *stream);	Gets the current position of the file pointer.

Library Functions (contd.) III

rewind	stdio.h	void rewind(FILE *stream);	Repositions the file pointer associated with stream to the beginning of the file.
feof	stdio.h	int feof(FILE *stream);	Tests whether the end-of-file flag is set for a given stream.
ferror	stdio.h	int ferror(FILE *stream);	Tests for an error indicator in reading from or writing to stream.
fclose	stdio.h	int fclose(FILE *stream);	Closes the specified stream.
perror	stdio.h	void perror(const char *string);	Prints an error message to stderr.
atoi	stdlib.h	int atoi(const char *string);	Converts string to an integer.
exit	stdlib.h	void exit(int status);	Ends a program normally.
system	stdlib.h	int system(const char *string);	Passes string to the system command analyzer.
malloc	stdlib.h	void *malloc(size_t size);	Reserves a block of storage.
calloc	stdlib.h	void *calloc(size_t num, size_t size);	Reserves storage space for an array of num elements, each of size size, and initializes the values of all elements to 0.
realloc	stdlib.h	void *realloc(void *ptr, size_t size);	Changes the size of a previously reserved storage block.
free	stdlib.h	void free(void *ptr);	Frees a block of storage.
strlen	string.h	size_t strlen(const char *string);	Calculates the length of string.
strcpy	string.h	char *strcpy(char *string1, const char *string2);	Copies string2 into string1.
strcmp	string.h	int strcmp(const char *string1, const char *string2);	Compares the value of string1 to string2.
strcat	string.h	char *strcat(char *string1, const char *string2);	Concatenates string2 to string1.
strchr	string.h	char *strchr(const char *string, int c);	Locates the first occurrence of c in string.
strncasecmp	string.h	int strncasecmp(const char *string1, const char *string2, size_t count);	Compares strings without case sensitivity.

Library Functions (contd.) IV

strncat	string.h	char *strncat(char *string1, const char *string2, size_t count);	Concatenates up to count characters of string2 to string1.
strncmp	string.h	int strncmp(const char *string1, const char *string2, size_t count);	Compares up to count characters of string1 and string2.
strrchr	string.h	char *strrchr(const char *string, int c);	Locates the last occurrence of c in string.
strspn	string.h	size_t strspn(const char *string1, const char *string2);	Returns the length of the initial substring of string1 consisting of characters contained in string2.
strstr	string.h	char *strstr(const char *string1, const char *string2);	Returns a pointer to the first occurrence of string2 in string1.
strtok	string.h	char *strtok(char *string1, const char *string2);	Locates the next token in string1 delimited by the next character in string2.
strtok_r	string.h	char *strtok_r(char *string, const char *seps, char **lasts);	Locates the next token in string delimited by the next character in seps. (Restartable version of strtok.)
signal	signal.h	void(*signal(int sig, void(*func)(int)))(int);	Registers func as a signal handler for the signal sig.

Example of usage of Library Functions

We can use the C functions `pow(power)` and `sqrt` to compute the root of a quadratic equation in x of the form $ax^2 + bx + c = 0$.

The two roots are defined as

$$root_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad root_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

```
/* Compute two roots, root_1 and root_2, for disc > 0.0 */
disc = pow(b,2) - 4 * a * c;
root_1 = (-b + sqrt(disc)) / (2 * a);
root_2 = (-b - sqrt(disc)) / (2 * a);
```

Top-Down Design and Structure Charts

Top-down design is a problem-solving method in which you first break a problem up into its major subproblems and then solve the subproblems to derive the solution to the original problem.

Structure chart is a documentation tool that shows the relationships among the subproblems of a problem.

Drawing Simple Diagrams

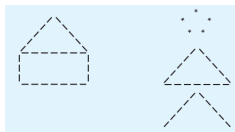


Figure 3: House and Stick Figure

Four basic components:

- a circle
- a base line
- parallel lines
- intersecting lines

Drawing Simple Diagrams (contd.)

DESIGN

To create the stick figure, you can divide the problem into three subproblems.

INITIAL ALGORITHM

- 1 Draw a circle.
- 2 Draw a triangle.
- 3 Draw intersecting lines.

ALGORITHM REFINEMENTS

Because a triangle is not a basic component, you must refine step 2, generating following subproblems:

Step 2 Refinement

- 2.1 Draw intersecting lines.
- 2.2 Draw a base.

Structure Chart for Drawing a Stick Figure

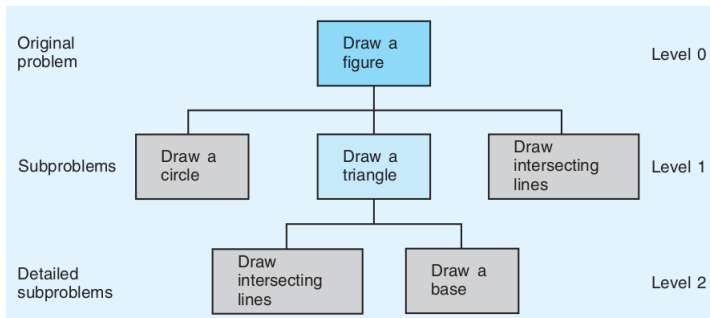


Figure 4: Structure Chart

User Defined Functions

Function Prototypes

return_type function_name (dummyargumentlist);

Example

```
oid sum(void); /*function with no argument and returning no value */  
int sum(void); /*function with no argument and returning a value */  
void sum(int, int); /*function with multiple arguments and returning no value */  
int sum(int, int); /*function with multiple arguments and returning a value */
```

More examples

```
nt sum(int x, int y);  
int sum(int *x, int *y);  
int * sum(int *x, int *y);
```

Function Definition

```
return_type function_name (formalargumentlist)  
{  
/*local variable declarations */  
/* processing statements */  
return (value);  
}
```

Example

```
int sum(int x, int y)  
{  
int sum;  
sum=x+y;  
return sum;  
}
```


Function Calling

variable = function_name (actual_argument_values);

Example

result = sum(10, 20);

or

printf(*“Result = %d”*, sum(10, 20));

A Complete Program using User Defined Function

```
#include <stdio.h>
int sum (int a, int b);  /* function prototype */
int main()
{
    int x, y, result;
    printf("Enter 2 integers: ");
    scanf("%d%d", &x, &y);
    result = sum(x,y);  /*calling function sum() */
    printf("Result = %d", result);

    return 0;
}
```

A Complete Program using User Defined Function (contd.)

```
/* Definition of sum() */  
int sum(int p, int q)  
{  
    int s;  
    s=p+q;  
  
    return s;  
}
```

Argument List Correspondence

- The number of actual arguments used in a cell to a function must be the same as the number of formal parameters listed in the function prototype.
- The order of arguments in the lists determines correspondence. the first actual argument corresponds to the first formal parameter, the second actual argument corresponds to the second formal parameter, and so on.
- Each actual argument must be of a data type that can be assigned to the corresponding formal parameter with no unexpected loss of information.

Quick-check Exercises (contd.)

1. Developing a program from its documentation means that every statement in the program has a comment. True or false?
2. The principle of code reuse states that every function in your program must be used more than once. True or false?

3. Write this equation as a C statement using functions `exp`, `log`, and `pow`:

$$y = (e^{n \ln b})^2$$

4. What is the purpose of a function argument?
5. Each function is executed in the order in which it is defined in the source file. True or false?
6. How is a function in a C program executed?
7. What is a formal parameter?
8. Explain how a structure chart differs from an algorithm.
9. What does the following function do?

```
void  
nonsense(void)  
{  
    printf("*****\n");  
    printf(" *   *\n");  
    printf("*****\n");  
}
```

Quick-check Exercises (contd.)

10. What does the following main function do?

```
int
main(void)
{
    nonsense();
    nonsense();
    nonsense();

    return (0);
}
```

11. If an actual argument of `-35.7` is passed to a type `int` formal parameter, what will happen? If an actual argument of `17` is passed to a type `double` formal parameter, what will happen?

Thank you

